LeakLens: Detecting Group and Temporal Data Leakage in Machine Learning Code by Linking Static Analysis to an Instrumented Run

Srijeet Prasad Banerjee **Adrish Sarkar** **Ankur Roy**

Sayandeep Dey **Soumyarik Mukherjee** **Tridib Chakraborty**

Department of Information Technology, Guru Nanak Institute of Technology, Kolkata, India

Abstract. Data leakage lets information from the evaluation data reach model training, so a reported score no longer predicts performance on new data. Existing tools that read machine learning code detect three kinds of leakage: preprocessing, overlap and multi-test. They do not detect group leakage, where the same patient, user or device appears on both sides of a split, and they cover temporal leakage only in part, although controlled studies find these to be the kinds that distort results most. We present LeakLens, a research prototype for Python scripts and Jupyter notebooks. A static pass based on abstract interpretation reports six leakage types and nominates the split call and the candidate entity or time column. An optional instrumented run then checks, fold by fold, whether entities or time periods really overlap at that call, with nothing declared by the user. For a located leak, LeakLens can apply a canonical fix to a copy of the pipeline, re-run it, and report the change in the pipeline's own metric. On the held-out half of our benchmark of injected leaks (79 files, 73 leaks), scored once under a protocol frozen beforehand, the static pass finds 59 leaks and the combined mode 65, with no false positive. On the same files, scored per file, the static analyser of Yang et al. is right for 20 of the 37 files that hold a leak of the three types it supports, against 33 for the static pass of LeakLens (exact McNemar test, $p < 0.001$). The run raises group detection from 3 of 7 to 7 of 7 by the matching rule, but only 3 findings name the true entity column. Given the same exported splits, four data-level checkers and a run-time checker find the group and time-order leaks well once the right column is named, and poorly otherwise: without a named column the best of them finds 3 of 8 shuffled time-series splits, which LeakLens finds 8 of 8 from the code. For impact estimates, a proxy-model tool was more accurate than LeakLens on group leaks in the held-out run, because LeakLens grouped by the wrong column. All results are on synthetic files written by the authors of the tool; no result on human-labelled real notebooks exists yet. We report the failures in detail, including a case study in which the tool found nothing in real code with known leakage.

Keywords: data leakage, static analysis, dynamic analysis, Jupyter notebooks, machine learning engineering, reproducibility

1. Introduction

A machine learning score is meaningful only if the model never saw the evaluation data during training. *Data leakage* is any path by which evaluation rows, or information that is not available at prediction time, influence training [1]. Leaky code runs without error and its tests pass. The only symptom is a score that is better than it should be.

Leakage is common. A survey of earlier reviews found it in 294 papers across 17 scientific fields; in one case study, complex models lost their reported advantage over logistic regression once the errors were corrected [2]. A static analysis of 107,603 public GitHub notebooks reported at least one leak in 29.6% of them [3].

Not all leaks do equal damage. Across 2,047 tabular datasets, fitting a scaler on all rows before splitting, the textbook example, changed AUC by at most 0.005 in all nine conditions tested, while leakage across group and time boundaries was invisible under random cross-validation [4]. In translational EEG research, only 17 of 63 surveyed deep learning studies clearly avoided placing segments of one subject in both training and test data, and in one experiment accuracy fell from 99.8% to 53.0% when subjects were held out properly [5].

The tools that read code look mostly at the first kind. The reference static analyser of Yang et al. [3] and the tools built on it [6, 7, 8] detect preprocessing, overlap and multi-test leakage. To our knowledge, no published static analyser detects group leakage from source code, and temporal leakage is covered in part by one recent LLM-based linter [9]. Tools that check the data itself can find date overlap [13] and group overlap [19, 18], but the user has to export the training and test tables and usually name the column, and the result does not point to a line of code. None of these tools reports how much a detected leak changes the user's own result.

This paper describes LeakLens[1], which addresses these three gaps. Its contributions are:

1. **Group leakage detected from source code**, with the entity inferred from column and variable names and from identifiers derived from file paths. Temporal leakage is detected through data flow. This is not a first for temporal leakage [9]; we treat it as an improvement on existing code-level checks.
2. **A link from static findings to an instrumented run.** The static pass nominates the split call and candidate columns; the run checks entity and time overlap at that call, per fold, with no user declaration.
3. **A per-finding impact estimate** obtained by re-running the user's own pipeline with a canonical fix.
4. **A benchmark of 175 files with 162 injected leaks** that includes group and temporal leakage, with a development and test split fixed before any detection rule was written.

We state the limits of the evidence at the outset. Every number in this paper is measured on synthetic files that we generated ourselves. A set of 150 real notebooks has been sampled but not yet labelled, so we make no claim about precision or recall on real code.

2. Background and Related Work

Taxonomies. Kaufman et al. [1] gave the first formal treatment of leakage. Kapoor and Narayanan [2] describe eight types, among them preprocessing on training and test data together,

[1] An unrelated package with the same name exists on PyPI. The name of this project may change before any release.

duplicates, temporal leakage and non-independence between training and test samples. Roth [4] measured severity by type in controlled experiments and found that the commonly taught type matters least.

Static analysis of ML code. Yang et al. [3] model data flow in Datalog and report three leakage types, with a precision of 97.6% and a recall of 67.8% on hand-labelled notebooks. NBLyzer and later abstract-interpretation work verify the absence of contamination between training and test data [10, 11, 12]. LeakageDetector wraps the analyser of Yang et al. in an IDE and adds quick fixes [6, 7]. Draoui et al. [8] combine static slicing with LLM prompts; their abstract reports F1 gains of 22% for preprocessing and 15% for overlap leakage over earlier work (we could read only the abstract). scicode-lint [9] asks a local LLM pattern questions about scientific code; two of its patterns concern temporal leakage (shuffled cross-validation on temporal data, and forward-looking shifts and centred windows). We found no group-leakage pattern among its definitions.

Checks on data and at run time. Deepchecks [13] checks duplicate rows and date overlap between training and test tables supplied by the user, and several recent packages add group overlap [19, 18]. splitguard [17] instruments splits and fits at run time and flags group overlap at the split line, but only for group identifiers the user registers. ArgusEyes [14] detects row overlap through provenance tracking in an instrumented run.

Measuring the effect. LeakHound [18] estimates inflation with a proxy model that it trains itself on the given split. mlwhatif [15] re-executes user pipelines with patches, for data-cleaning questions rather than leakage. Leakly [21] re-runs a whole pipeline on permuted labels; a score above chance suggests leakage somewhere, without locating it or applying a fix. LeakLens differs in that the estimate belongs to one located finding and is measured on the user's own pipeline and metric.

Table 1 compares LeakLens with the analyser of Yang et al., the closest prior work and the engine inside the LeakageDetector tools.

3. Leakage Taxonomy

LeakLens reports six types (Table 2). The first three correspond to the types of Yang et al. [3]. Group and temporal leakage correspond to non-independence and temporal leakage in the taxonomy of Kapoor and Narayanan [2], and target-derived features to their illegitimate features.

A group finding depends on the claim being made. A model meant to be used on the same patients it was trained on may legitimately share patients across a split. The tool cannot know the claim, so group findings carry a confidence level and can be suppressed inline with a stated reason.

4. Design of LeakLens

Figure 1 shows the three stages. The static stage never runs the analysed code. The other two execute it and are optional.

4.1 Static pass

Notebooks are read in execution order when every code cell has a unique execution count, and in cell order otherwise. The analyser then makes one pass of abstract interpretation over the flattened module. Each variable receives an abstract value that records what the analysis knows: the origin of a data frame, its known columns, which statistics have been computed over which rows, whether rows have been copied or resampled, and the training or evaluation role of the result of a split.

Rules fire at *sinks*. At a split point (`train_test_split`, a splitter's `split`, a cross-validation helper, a search object, or a manual random sample with its complement) the analyser checks for group and temporal leakage and reports every statistic or copied row that reached the data being split. At a model fit it reports statistics computed on rows that include the evaluation side. At a prediction or score it checks whether the model was fitted on rows that include the scored rows. A selection over scores computed in a loop gives a multi-test finding.

For group leakage, the evidence for an entity is a column or variable name, or an identifier derived from a file path (for example a subject code split from a recording's file name). Names are matched against two lists. Strong names such as `patient`, `subject` or `user` can give a finding of medium confidence. Weak names such as `customer`, `id` or `visit` give low confidence only. Listing 1 shows a file-path case.

The analyser prefers missing a leak to inventing one. A call it does not model is treated as depending on all its inputs, and anything it cannot resolve produces no finding. Loops are analysed once, functions are inlined to a fixed depth, and classes are opaque. pandas, NumPy, scikit-learn [16] and imbalanced-learn are modelled by name; all catalogues and thresholds are in configuration files.

4.2 Instrumented run

Static analysis cannot see an entity column that the code never mentions. A script may load a table with a `patient_id` column and split it at random without naming that column.

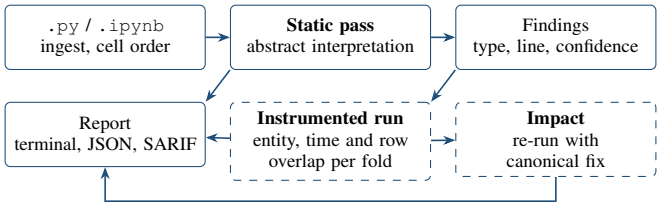


Figure 1. Stages of LeakLens. Dashed stages execute the code and are optional.

```
recordings = {f"sub-{i:02d}_run-1.npy": ... }
windows, labels, subjects = [], [], []
for fname, signal in recordings.items():
    subject = fname.split("_")[0]
    for w in signal.reshape(-1, 500):
        windows.append([w.mean(), w.std()])
        labels.append(...)
        subjects.append(subject)
X, y = np.array(windows), np.array(labels)
print(cross_val_score(LogisticRegression(), X, y, cv=5).
     mean())

line 16  [high/medium] group (LL-GRP-003)
  Rows are split without regard to `subjects`; the same
  subjects can be in training and evaluation rows.
  fix: use GroupKFold with groups=`subjects`
```

Listing 1. Group leakage through cross-validation with the default splitter, and the finding the static pass reports for the full file (code and message shortened).

Table 1. Design comparison between the static analyser of Yang et al. and LeakLens. Entries for Yang et al. are taken from their paper and from our own runs of their released tool.

Aspect	Yang et al. [3]	LeakLens (this work)
Venue and year	ASE 2022	Research prototype, 2026
Analysis technique	Static data-flow analysis written in Datalog (Soufflé), with type inference	One-pass abstract interpretation over the Python syntax tree, linked to an optional instrumented run
Runs the analysed code	No	Optional (dynamic and impact modes only)
Input	Python scripts; notebooks have to be converted first	`.py` and `.ipynb` directly, in execution order when it is recorded
Leakage types	Preprocessing, overlap, multi-test	The same three, plus group, temporal and target-derived features
Group (entity) leakage	Not a type it claims	Yes: entity inferred from names and file paths, confirmed on the data at run time
Temporal leakage	Not a type it claims	Yes: non-chronological splits and forward-looking features
Location reported	The affected evaluation line (as observed in our runs)	The line that causes the leak, with a suggested fix
Confidence and severity	Not reported	Both, per finding
Impact on the score	Not estimated	Re-runs the pipeline with a canonical fix; six kinds of fix so far
Output and integration	HTML report, command line	Terminal, JSON, SARIF; pre-commit hook; GitHub Action
Dependencies	Soufflé, a pyright fork, Python 3.8, Linux	Python 3.12 with NumPy, pandas, scikit-learn
Evaluation data	100 hand-labelled real notebooks (94 analysed) and 107,603 public GitHub notebooks	175 synthetic files with injected leaks, written by us; 150 real notebooks sampled but not yet labelled
Accuracy reported by the authors	Precision 97.6% (40/41), recall 67.8% (40/59), real notebooks	Precision 1.00, recall 0.81 (static) and 0.89 (with the run), own injected TEST split
Are the two accuracy rows comparable?	No. They were measured on different data with different labelling. Table 4 gives the only like-for-like measurement.	
Time per file	Mean 3.23 s (their paper); median 6.2 s on our files (Kaggle CPU)	Static: median 2.6 ms. Instrumented run: median 1.3 s (laptop)
Large-scale study	29.6% of 107,603 notebooks flagged	None yet
Licence	MIT	MIT

The dynamic pass handles this case. It runs the script in a child process with scikit-learn entry points wrapped: `train_test_split`, the `split` method of every splitter, the cross-validation helpers, estimator `fit`, `predict` and `score`, and the metric functions.

At each split, and for each fold, the wrapper records three facts. Rows are hashed by value, after rounding numbers, to find identical rows on both sides. Entity columns are looked up in the data frames of the script's own namespace that are row-aligned with the data being split; a column is reported when its values repeat and enough evaluation rows belong to an entity that also has training rows. If a time column is found, the wrapper checks whether training rows are dated after the earliest evaluation row.

These facts are linked to the static findings by call site. A finding ends up *confirmed*, *refuted* or *not checked* (no run-time check exists for that rule). Findings that only the run can make, such as duplicate records already present in the raw data, are added.

4.3 Impact estimation

For a located finding, LeakLens re-runs the script with one call site changed and nothing else. Six kinds of fix are implemented: duplicated records are removed before a split; a row-level split becomes a group-aware one on the detected column; a non-chronological split becomes a chronological one; evaluation rows are removed from a model fit; a transformer fitted on all rows is fitted on the training rows of a hold-out split; and a feature is removed. The estimate is the script's printed metric in the original run minus the metric in the fixed run. Findings without an automatic fix, or whose fixed run fails, are reported as "impact unknown" and are never dropped. When the fixed run takes a different path through the code, the estimate is marked unreliable.

4.4 Running untrusted code

The dynamic and impact modes execute the analysed file. For notebooks from the internet, LeakLens runs them only through a sandbox inside an offline cloud session: a child process with an allow-listed environment, a throwaway working directory, a process-group timeout and an unprivileged user. Each layer answers a problem we measured. The platform we use (Kaggle) injects session tokens into every job, and a child process running as root could read them from `/proc` even with a clean environment. After the privilege drop, a scan from the child found the token in none of its ancestor processes, while a control scan as root found it in two.

4.5 Reporting and integration

Each finding carries the file, line range, type, rule identifier, confidence, severity, evidence, a suggested fix and, when measured, the metric before and after the fix. Output formats are

Table 2. The six leakage types and the signal used by each pass.

Type	Static signal	Run-time check
Preprocessing	A transformer, selector or encoder is fitted on rows that later reach the evaluation side	The fit included evaluation rows
Overlap	Resampling or concatenation before the split; evaluation rows flow into a fit	Identical rows on both sides of the split
Multi-test	Several candidates scored on the same evaluation rows feed a selection	Same rows scored repeatedly
Group	A row-level splitter or default CV on data with a candidate entity	The same entity value on both sides
Temporal	A shuffled split on time-ordered data; forward-looking feature	Training rows dated after evaluation rows
Target-derived	A feature computed from the label	Very high single-feature score

terminal text, JSON with a published schema, and SARIF, which GitHub code scanning accepts. A pre-commit hook and a GitHub Action are provided. The tool is deterministic and uses no language model. Its test suite has 373 tests, including property-based tests that the analyser terminates and gives identical output on repeated runs; on a laptop 371 pass and the 2 sandbox tests that need root are skipped (they pass as root in the cloud job).

5. Benchmark and Evaluation Protocol

5.1 Injected benchmark

No public labelled dataset includes group and temporal leakage, so we built one. Thirteen clean base pipelines in three families (tabular, time series and grouped data such as multi-visit patients and EEG-style windows) are mutated by 18 operators. Seventeen inject a leak, for example a scaler fitted before the split, oversampling before the split, a shuffled split on time-ordered data, or a row-level split on grouped data. One produces a *trap*: code that resembles a leak but is correct. Six files combine two leaks. The result is 175 files (94 scripts and 81 notebooks) with 162 labelled leaks, 13 clean files and 6 traps. The lines that carry each leak are known by construction, and every leaky file has a corrected version, so the true inflation of its metric can be measured.

Temporal leaks are divided into three categories: *common* (for example an explicit shuffled split), *differentiating* (cases that pattern-based checks do not cover, such as the default shuffle of `train_test_split`) and *hard* (for example a time column with a name that does not suggest time).

The benchmark is split by base pipeline into DEV (7 pipelines, 96 files, 89 leaks) and TEST (6 pipelines, 79 files, 73 leaks). The assignment was committed before any detection rule existed. Rules and thresholds were tuned on DEV only.

5.2 Real notebooks

We sampled 150 public Kaggle notebooks in three query-based strata of 50 and split them by author into DEV (71) and TEST (79). They are to be labelled by two people who do not see any tool output, with agreement reported as Cohen's kappa. **This labelling has not been done.** The static pass was run on the 71 DEV notebooks only to check robustness: it analysed all of them without a crash, at a median of 4 ms per notebook. What it flagged there is not a result, because there are no labels.

5.3 Protocol

The protocol was written after DEV results had been seen and frozen before any TEST result existed. A finding matches a label when it is in the same file and cell, its lines overlap the labelled lines, and its type is the labelled type or a listed alternative. Matching is one-to-one. An unmatched finding is a false positive and an unmatched label is a miss. The primary operating point is findings of medium or higher confidence. Recall is reported per type with 95% percentile bootstrap intervals over files (2,000 resamples); we do not report a single pooled score as the headline. TEST is scored once per release, and any later change is logged and its result labelled as a second look.

For impact estimation, the true inflation of an injected file is the metric of the leaky file minus the metric of its corrected version, averaged over five data seeds. The generators were designed, and for group leakage adjusted after a first measurement, so that each leak type can change the metric. These inflation values are therefore used only to score detection and the accuracy of impact estimates. They are not evidence of how severe any leakage type is in practice.

5.4 Baseline

We built the released analyser of Yang et al. [3] from source in a Linux cloud job and ran it with its default command. Its packaged Datalog engine no longer installs on the current image, so the engine was built from source at version 2.4.1; this is a deviation from the released setup. The tool accepts Python scripts, so notebooks were given to it in a plain-script rendering. It reports the affected evaluation line and not the line that causes the leak, so the line-overlap rule cannot be applied to it. For this comparison only, both tools are scored per file and type: a tool is right about a type in a file when it reports that type and the file has a labelled leak of that type. This rule was fixed before any output of the baseline on TEST existed.

Data-level and run-time tools. leakcheck-ml [19], LeakHound [18], dataleaks [20] and Deepchecks [13] take a training and a test table, not code. The instrumented run of LeakLens can export, at every split call and for up to five folds, the training and evaluation tables the pipeline actually used (every row-aligned column of the script's data frames, with the label column removed and the target kept separately). Each tool received every exported pair twice: with its defaults, and with the true entity or time column named through its own documented parameter. A detection is placed at the split call's line (protocol). splitguard [17] instruments a run itself; it was run as its documentation prescribes, and again with the true group declared through its `mark_groups` call. Each tool ran in its own environment at the version cited; Deepchecks needs an older Python stack (3.11, NumPy 1.26). For all baselines, a finding is one per file, line and type, as for LeakLens. These tools were run after the single TEST run of LeakLens, in separate jobs; they were not tuned, so this cannot inform LeakLens.

Table 3. Detection on the injected TEST split (79 files, 73 leaks), scored once. Findings of medium or higher confidence. Recall with 95% bootstrap interval over files. No false positive in either mode.

		Static		Static + run	
Type	**Leaks**	**Found**	**Recall**	**Found**	**Recall**
Preprocessing	22	21	0.95 (0.84–1.00)	21	0.95 (0.84–1.00)
Overlap	9	7	0.78 (0.44–1.00)	9	1.00 (1.00–1.00)
Multi-test	6	5	0.83 (0.43–1.00)	5	0.83 (0.43–1.00)
Group	7	3	0.43 (0.00–0.83)	7*	1.00*
Temporal	25	20	0.80 (0.62–0.95)	20	0.80 (0.62–0.95)
Target-derived	4	3	0.75 (0.00–1.00)	3	0.75 (0.00–1.00)
All types	73	59	0.81	65	0.89

*By the matching rule, which checks type and line. Only 3 of the 7 findings name the true entity column, so the strict figure is 0.43.

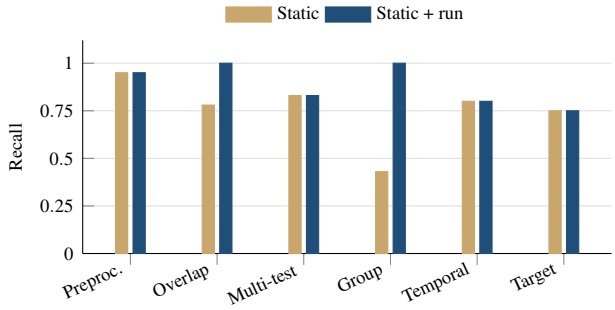


Figure 2. Recall by type on injected TEST. The group bar for the combined mode follows the matching rule; with the true entity column required it is 0.43.

Not run. scicode-lint [9] needs a GPU with at least 16 GB and native FP8 support in its published setup. On the GPUs available to us (two NVIDIA T4, no FP8), its pinned model server (vLLM 0.18.0) failed to start; a fallback server (llama.cpp with the same model at 8-bit) answered but produced no result within 20 minutes per file for any of the 29 files it attempted. We report it as not run, not as zero. LeakCheck [8] needs a paid LLM interface and an access-restricted package.

6. Results

6.1 Detection on the held-out split

Table 3 gives the single TEST run. There was no false positive in either mode and no finding on any of the 9 files without a leak. The static pass found 59 of 73 leaks (0.81) and the combined mode 65 (0.89). Figure 2 shows recall by type.

The run confirmed 43 static findings, refuted none, could not check 25 (no run-time check exists for forward-looking features or pandas statistics) and made 2 findings of its own, both duplicated records. By temporal category, the tool found 5 of 5 common, 7 of 7 differentiating and 8 of 13 hard leaks. On DEV, where the rules were tuned, the static pass found 80 of 89 leaks and the combined mode 89 of 89; the drop from DEV to TEST is real.

6.2 What went wrong on TEST

Right line, wrong column. In a patient-visits pipeline the static pass made a low-confidence guess on a column named `visit`. The confirmation step looked only at nominated columns, found that visit numbers occur on both sides of the split, as they always do, and raised the confidence. It never examined `patient_id`. The matching rule does not check the column, so Table 3 counts these four findings as correct. They are not: the entity is wrong, and so is the impact estimate that follows from it.

Table 4. Yang et al. and LeakLens (static mode) on the 79 injected TEST files, scored per file and type. "Found": files with this leak in which the tool reports the type. "False": files where it reports the type wrongly.

		Yang et al. [3]		LeakLens (static)	
Leakage type	**Files**	**Found**	**False**	**Found**	**False**
Preprocessing	22	10	1	21	0
Overlap / duplicates	9	7	0	7	0
Multi-test	6	3	0	5	0
Group†	7	0	0	3	0
Temporal†	24	3	0	19	0
Target-derived†	4	0	0	3	0

†Not a type the baseline claims to detect. It is credited under temporal in 3 files where it reports whole-series scaling as preprocessing leakage. Temporal has 24 files for 25 leaks because one file holds two.

A slice mistaken for a split. One pipeline drops its last row with `data.iloc[:-1]`. The analyser treated this as the training side of a manual split. Five temporal leaks were reported at low confidence only and one multi-test leak was missed.

An unresolved column filter. A feature list built with a `startswith` filter could not be resolved, so two leaks in that pipeline were not reported.

6.3 Second look

These three faults were then fixed and the change logged. DEV was re-run and came out unchanged, and TEST was run again: 67 of 73 with the static pass and 73 of 73 with the run, with no false positive, and all group findings now naming the true entity. The fixes were written knowing which TEST files they repair, so this is not a held-out result and Table 3 remains the result of record. The injected benchmark is now saturated on both halves and cannot measure further progress.

6.4 Comparison with Yang et al. on the same files

Table 4 gives the per-file comparison. On the three types the baseline supports, it found 20 of 37 files with one false alarm, and the static pass of LeakLens found 33 of 37 with none. Both tools analysed all 79 files and reported nothing on the 9 leak-free files. On the 37 file-and-type pairs, LeakLens alone was right in 14, the baseline alone in 1, both in 19 and neither in 3; an exact McNemar test gives $p = 0.001$. The difference comes from preprocessing leakage (12 against 1); on overlap the two tools agree on every file.

This comparison favours LeakLens, and should be read with that in mind. Neither tool was tuned on these files, but the files were generated by our own operators, with the kinds of leak LeakLens was built for, in a style that may differ from the notebooks the baseline was designed on. The figures its authors report on their own hand-labelled notebooks (precision 97.6%, recall 67.8%) are a different measurement and cannot be set against Table 4. A fair comparison needs the labelled

Table 5. Leaks at a split call, injected TEST. T1: non-chronological splits of time-ordered data. Grp: group splits by the matching rule; Grp, col.: with the true entity column named. Dup.: duplicated raw records. Clean: leak-free files with any finding. LeakLens: first look, result of record.

Tool	Column given	T1 /8	Grp /7	Grp, col. /7	Dup. /2	Clean /9
LeakLens (static + run)	none needed	8	7	3	2	0
leakcheck-ml	none (default)	3	4	4	2	0
	true column named	7	7	7	2	0
leakhound-ml	none (default)	0	0	0	2	0
	true column named	7	7	7	2	0
dataleaks	none (default)	0	7	4	2	2
	true column named	0	7	7	2	2
deepchecks	none (default)	0	0	0	2	0
	true column named	7	0	0	2	0
splitguard	none (default)	0	0	0	0	0
	true column named	0	7	7	0	0

real notebooks.

6.5 Leaks at a split: data-level and run-time tools

Table 5 compares the tools on the TEST leaks that sit at a split call, the only ones a data-level tool can see: 8 non-chronological splits of time-ordered data, 7 group splits and 2 sets of duplicated records already present in the raw data. Without a named column, leakcheck-ml finds 4 of 7 group splits, all with the right column, and 3 of 8 time splits; LeakHound, Deepchecks and splitguard find none of either. Once the true column is named, leakcheck-ml and LeakHound find 7 of 8 and 7 of 7. dataleaks reaches 7 of 7 group splits under the matching rule without a hint, but names an unrelated column in 3 of them and reports a group overlap in 2 of the 9 leak-free files; it has no check for the order of a split in its default configuration. Deepchecks has no group check and finds the time splits only when told the time column. splitguard finds group splits only when the group is declared, and in this benchmark it does not see the held-out rows of a split taken with `next(splitter.split(...))`, so it misses later fits on them. All data-level tools find the duplicated records.

LeakLens finds 8 of 8 time splits without being told any column. On group splits it is weaker than the table's second column suggests: by the matching rule 7 of 7, but only 3 name the true entity, against 4 for leakcheck-ml without a hint. Every tool in the table except LeakLens also needs the user to export the tables, which the instrumented run of LeakLens did for them here.

6.6 Impact estimates

On TEST an estimate could be produced for 34 of 70 located leaks: all 7 group findings, 3 of 3 target-derived, 7 of 9 overlap, 10 of 21 preprocessing, 7 of 25 temporal and none of the 5 multi-test findings. The rest were reported as unknown with the reason. The instrumented script printed exactly the plain script's metric in all 395 runs.

For 26 single-leak findings with a bounded metric, averaged over five data seeds, the mean absolute error against the true inflation was 0.035 and the rank correlation 0.58. The estimate was exact where the fix is the same operation as the benchmark's corrected file, which is a weak test. It was wrong in direction for hold-out group splits (-0.084 estimated against a true $+0.212$ AUC), because the fix grouped by the wrong column. In the second look, with the column fault repaired, the mean absolute error was 0.0006 and the rank correlation 0.97; the same caveat applies as for detection.

Table 6. Impact estimates against the true inflation, injected TEST, single-leak files, seed 0. N: findings. Comp.: findings whose pipeline reports AUC or R^2, the metrics LeakHound reports. Cells: number of estimates (mean absolute error). LH: LeakHound with its defaults; LH, col.: with the true column named. LeakLens: first look.

Type	N	Comp.	LeakLens	LH	LH, col.
group	6	6	6 (0.158)	0 (–)	6 (0.075)
overlap	6	6	6 (0.000)	2 (0.043)	2 (0.043)
preproc.	8	8	8 (0.000)	0 (–)	0 (–)
target-der.	3	3	3 (0.000)	0 (–)	0 (–)
temporal	6	3	3 (0.000)	0 (–)	3 (0.059)

Table 6 compares these estimates with LeakHound's, which trains its own model on the exported split and compares it with a group-aware or chronological split it constructs. LeakHound reports AUC or R^2, so only pipelines with those metrics are compared, on data seed 0. Where both tools gave an estimate (11 findings), LeakHound's mean absolute error was 0.065 and LeakLens's 0.086: LeakLens was worse on the hold-out group splits, where it grouped by the wrong column and got the sign wrong, and exact on duplicated records and time splits, where LeakHound was off by 0.01 to 0.08. With the column fault repaired (second look, not held out) LeakLens's error on the same 11 findings was 0.020. LeakHound gave no estimate for preprocessing or target-derived findings, and none at all without a named column except for duplicated records. The near-zero preprocessing errors carry little information: the true inflation of those leaks is close to zero.

6.7 Running time

The static pass took a median of 2.6 ms per file and the instrumented run a median of 1.3 s (maximum 12.7 s) on a laptop. The baseline took a median of 6.2 s per file in a cloud CPU job. The machines differ, so these figures indicate scale only.

6.8 A negative case study

We ran the static pass on a separate EEG seizure-detection codebase written by the first author. It has three evaluation regimes whose leakage status is known: random windows, a within-patient split and leave-one-patient-out. The tool reported nothing in any of its 78 files. That is correct for the third regime and a miss for the first two. The code splits with hand-written NumPy masks and by position over lists of custom objects, and subject identity is an attribute of an object. There is no scikit-learn splitter, data frame or column name for the analyser to model, and the dynamic pass, which wraps scikit-learn entry points, would not see these splits either. No rule was changed after this result.

7. Limitations and Threats to Validity

No result on real code. All detection and impact figures come from our own injected benchmark. The tool and the benchmark have the same authors, and TEST uses different pipelines but the same mutation operators as DEV. The results

measure generalisation across pipelines, not across the ways people actually write leaks. A precision of 1.00 on short synthetic files will not hold on real notebooks.

Small samples. TEST has 4 target-derived, 6 multi-test and 7 group leaks. The intervals are wide.

Blindness. While the rules were being written, the column names of the TEST pipelines were seen by mistake. No TEST code, label or operator was read, but TEST group results are not blind to entity names.

Heuristic entity inference. Evidence for an entity is a name, repetition, or a file path. A name can mislead in both directions, and the first TEST run shows the consequence.

Coverage of code styles. The static pass models pandas, NumPy and scikit-learn by name. Hand-written splits, custom classes and deep learning frameworks are largely outside it, as the case study shows.

Impact estimates. Automatic fixes exist for six situations, covering about half of the located leaks. A group-aware or chronological split changes which rows are evaluated, so the estimate is the change under the canonical fix, not a causal decomposition.

Baselines. The analyser of Yang et al. was scored under a looser rule than the protocol's. The data-level tools were given tables exported by our own instrumented run, and ran in jobs after the single TEST run of LeakLens. scicode-lint and LeakCheck could not be run.

8. Conclusion and Future Work

LeakLens detects six types of data leakage in Python machine learning code, including group leakage from source code, links static findings to facts observed in an instrumented run, and estimates the effect of a located leak by re-running the pipeline with a fix. On a held-out set of injected leaks it found 81% of leaks statically and 89% with the run, with no false positive, and it found more leaks than the reference static analyser on the three types they share ($p = 0.001$). Against data-level tools given the same splits, it was the only tool that found the time-order leaks without being told the time column, and it was matched or beaten on group leaks by a tool that was told the entity column. The same evaluation exposed a confirmation fault that named the wrong entity in four of seven group findings, and a case study on real code showed that hand-written splits are invisible to it.

The next step is the one that decides whether these results mean anything outside our benchmark: labelling the 150 sampled notebooks and scoring LeakLens and the baselines on them. After that we plan to run scicode-lint on hardware with FP8 support, to extend the analysis to manual and framework-specific splits, and to measure how often each leakage type occurs in public notebooks and how much it changes their reported scores.

Data and Code Availability

The source code, benchmark generators, frozen protocol, evaluation change log and the output of every run cited here are in the project repository: `https://github.com/bytes06runner/LeakLens`.

Acknowledgement

The implementation was written with the help of an AI coding assistant. The authors made the design and protocol decisions and reviewed the results reported here.

References

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012. doi:10.1145/2382577.2382579.

[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 100804, 2023. doi:10.1016/j.patter.2023.100804.

[3] C. Yang, R. A. Brower-Sinning, G. A. Lewis, and C. Kästner, "Data leakage in notebooks: static detection and better processes," in *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2022. doi:10.1145/3551349.3556918.

[4] S. Roth, "Which leakage types matter? A quantitative landscape across 2,047 benchmark datasets," arXiv:2604.04199, 2026.

[5] G. Brookshire *et al.*, "Data leakage in deep learning studies of translational EEG," *Front. Neurosci.*, vol. 18, 1373515, 2024. doi:10.3389/fnins.2024.1373515.

[6] E. A. AlOmar, C. DeMario, R. Shagawat, and B. Kreiser, "LeakageDetector: an open source data leakage analysis tool in machine learning pipelines," arXiv:2503.14723, 2025.

[7] Truong *et al.*, "LeakageDetector 2.0: analyzing data leakage in Jupyter-driven machine learning pipelines," arXiv:2509.15971, 2025.

[8] T. Draoui, M. W. Mkaouer, and C. Newman, "Improving data leakage detection in machine learning notebooks through static slicing and structured LLM prompts," *Proc. ACM Softw. Eng.*, vol. 3, no. FSE, 2026. doi:10.1145/3808199.

[9] Samsonau, "scicode-lint: detecting methodology bugs in scientific Python code with LLM-generated patterns," arXiv:2603.17893, 2026.

[10] P. Subotić, L. Milikić, and M. Stojić, "A static analysis framework for data science notebooks," in *Proc. 44th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022. doi:10.1145/3510457.3513032.

[11] P. Subotić, U. Bojanić, and M. Stojić, "Statically detecting data leakages in data science code," in *Proc. 11th ACM SIGPLAN Int. Workshop on the State of the Art in Program Analysis (SOAP)*, 2022. doi:10.1145/3520313.3534657.

[12] F. Drobnjaković, P. Subotić, and C. Urban, "An abstract interpretation-based data leakage static analysis," in *Theoretical Aspects of Software Engineering (TASE)*, LNCS, Springer, 2024. doi:10.1007/978-3-031-64626-3_7.

[13] S. Chorev *et al.*, "Deepchecks: a library for testing and validating machine learning models and data," *J. Mach. Learn. Res.*, vol. 23, no. 285, 2022.

[14] S. Schelter, S. Grafberger, S. Guha, B. Karlaš, and C. Zhang, "Proactively screening machine learning pipelines with ArgusEyes," in *Companion of the 2023 Int. Conf. Management of Data (SIGMOD)*, 2023. doi:10.1145/3555041.3589682.

[15] S. Grafberger, P. Groth, and S. Schelter, "Automating and optimizing data-centric what-if analyses on native machine learning pipelines," *Proc. ACM Manag. Data (SIGMOD)*, 2023. doi:10.1145/3589273.

[16] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[17] T. Aiello, "splitguard," software, version 0.1.0, 2026. `https://github.com/Tommasoaiello13/splitguard`.

[18] J. Gomes, "LeakHound," software, version 0.5.0, 2026. `https://github.com/happyhellpt/leakhound`.

[19] "leakcheck-ml," software, version 0.3.0, 2026. `https://github.com/Sofienguy1/leakcheck`.

[20] K. Rajput, "DataLeaks," software, version 0.1.2, 2026. `https://github.com/KAVYA-29-ai/Dataleaks`.

[21] DeMONLab-BioFINDER, "Leakly: leakage checks for machine-learning pipelines using permutation tests," software, version 0.1.2, 2026. https://github.com/DeMONLab-BioFINDER/Leakly.